Instituto Tecnológico Superior de Xalapa

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

ING. EN SISTEMAS COMPUTACIONALES

3° Semestre | Grupo: B

ESTRUCTURA DE DATOS

VAZQUES HERRERA VICTOR HUGO

UNIDAD 5
TAREA 1

PRESENTA

257O00081
Brayan Mejia Mora

15 DE NOVIEMBRE DEL 2026

Implementación del Algoritmo de Ordenamiento por Burbuja

El algoritmo de ‘Ordenación por Burbuja’ (o Bubble Sort), es uno de los algoritmos más simples de ordenación que puedes encontrar en los libros y en internet; es con el que la mayoría aprende paso a paso cómo se realiza la ordenación de una lista de números en un arreglo, por ello es bueno realizar un análisis de cómo funciona. Este algoritmo, aunque cumple con el trabajo, no lo hace de forma perfecta y tiene defectos. Por supuesto, no es el único algoritmo de ordenación; hay una gran variedad de otros algoritmos que hacen lo mismo, la diferencia es que buscan minimizar la cantidad de tiempo y/o memoria.

Se usa principalmente con fines educativos. En producción, es preferible Quick Sort o Merge Sort.

Ubicación del código

El siguiente link corresponde justamente a la carpeta donde se encuentra el código de BubbleSort, en un repositorio de GitHub:

https://github.com/brianlewyn/data-structure-tec-3th-semester/tree/main/Unidad5/BubbleSort

¿Qué es el “Ordenamiento por Burbuja”?

Según Singh (2026), de la plataforma AlgoMaster, dedicada a impartir cursos online sobre programación, el algoritmo se define de la siguiente manera:

> Bubble sort is a comparison-based sorting algorithm that works by repeatedly stepping through the array, comparing adjacent elements, and swapping them if they are in the wrong order. Each pass through the array moves the largest unsorted element to its correct position at the end. (AlgoMaster, 2026)

En síntesis, el ordenamiento por burbuja es un algoritmo de ordenación que se basa en comparaciones repetitivas y sucesivas a través de un arreglo. Dichas comparaciones se realizan entre elementos adyacentes (es decir, aquellos que están juntos, uno al lado de otro). La lógica consiste en aplicar una regla de intercambio: si el elemento de la izquierda es mayor que el de la derecha, ambos se intercambian para acercarlos a su posición correcta. Mediante repeticiones sucesivas (cuyo número depende de cuántos elementos contenga el arreglo), el elemento más grande se desplaza hasta el final en cada pasada, logrando que, al finalizar, el arreglo quede completamente ordenado.

¿Por qué tiene el nombre de “Burbuja”?

Según AlgoMaster (2026), la historia deriva de las bebidas carbonatadas, donde las burbujas más grandes suben a la superficie más rápido; es decir, los elementos más grandes son los que suben más rápido (o también, son los que se mueven hacia adelante hasta el final del arreglo).

¿Cómo funciona el algoritmo de ordenación?

Cabe mencionar que los siguientes pasos no siguen el algoritmo tradicional, sino que presentan una versión optimizada. Esta variante incorpora una bandera (flag) para evitar recorridos innecesarios cuando el arreglo ya está ordenado, mejorando así su eficiencia en el mejor de los casos.

1. Compara cada par de elementos adyacentes
2. Intercámbialos si el elemento izquierdo es mayor
3. Después de cada pasada, el elemento más grande no ordenado queda en su posición final
4. Repite hasta que no se necesiten más intercambios
5. Detente cuando no se realice ningún intercambio durante una pasada
   (esto significa que el arreglo está ordenado).

Análisis de complejidad

Para analizar la complejidad del código, se requirió agregar unas líneas de código extra para medir la eficiencia del mismo.

Al método ordenar, se le agregaron dos contadores: uno para medir la cantidad de comparaciones y otro para medir la cantidad de intercambios. Antes de terminar la función, se imprimen estos valores por consola.

```java
public static void ordenar(int[] arr) {
    int n = arr.length;
    boolean intercambiado;

    int nComparaciones = 0;
    int nIntercambios = 0;

    for (int i = 0; i < n - 1; i++) {
        intercambiado = false;

        for (int j = 0; j < n - i - 1; j++) {

            nComparaciones++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                intercambiado = true;
                nIntercambios++;
            }
        }

        if (!intercambiado) {
            break;
        }
    }

    System.out.printf("\nNúmero de comparaciones: %d", nComparaciones);
    System.out.printf("\nNúmero de intercambios: %d\n\n", nIntercambios);
}
```

Además, en el archivo main, se agregaron arreglos extra para medir la complejidad en el mejor caso, en un caso promedio y en el peor caso.

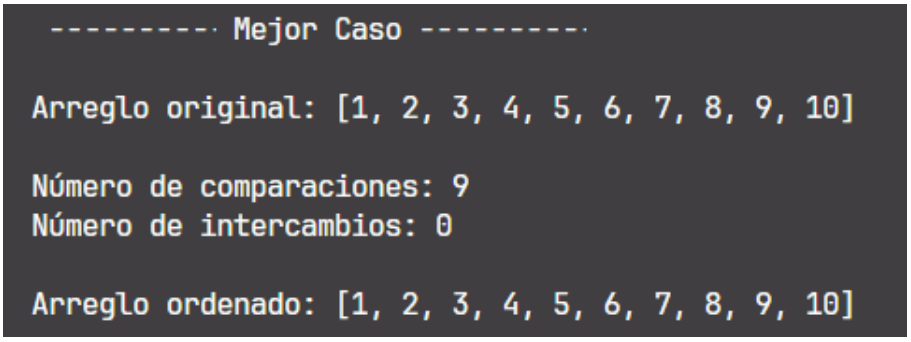
```java
int[] mejorCaso = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
imprimirPruebaDeArreglo("Mejor Caso", mejorCaso);

int[] casoPromedio = {6, 1, 10, 5, 8, 3, 2, 9, 7, 4};
imprimirPruebaDeArreglo("Caso Promedio", casoPromedio);

int[] peorCaso = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
imprimirPruebaDeArreglo("Peor Caso", peorCaso);
```

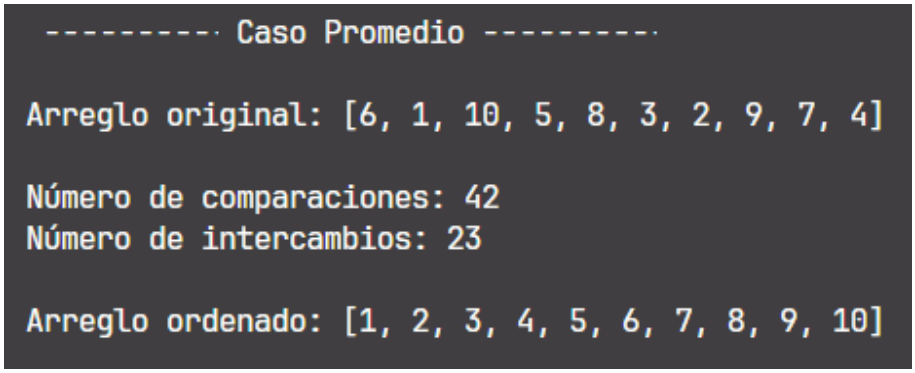
Algoritmo en el mejor caso O(n)

Si el arreglo ya esta ordenado, cuando se ejecute el algoritmo intentará comprobar cada par adyacente, si no hay intercambios al finalizar la primera pasada, el algoritmo se detiene. Lo que resulta en n-1 comparaciones y cero intercambios.



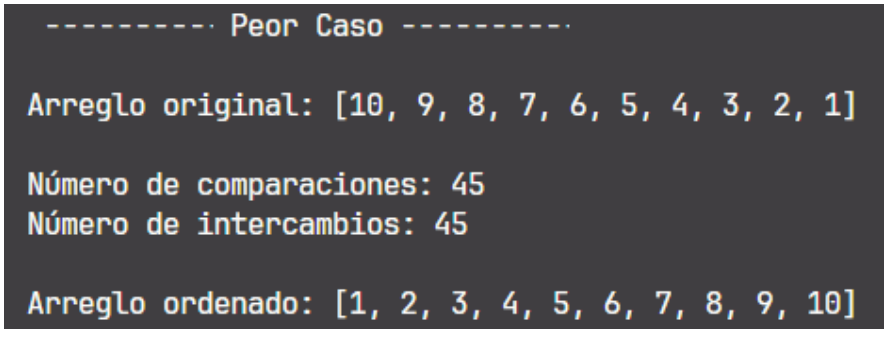
Algoritmo en caso promedio O(n^2)

Los elementos están en orden aleatorio. En promedio, un parte aún necesita intercambiar sus pares, lo que conlleva a que siga siendo una estructura en forma de bucle, es decir, es una complejidad cuadratica.



Algoritmo en el peor caso O(n^2)

Los elementos estan ordenados en orden contrario. Es decir, cada par requiere un intercambio. Esto resulta exactamente n(n-1)/2 comparaciones e intercambios.



Conclusión

En conclusión, el algoritmo de ordenamiento Bubble Sort representa el punto de partida ideal para comprender cómo funcionan las comparaciones e intercambios en un arreglo, siendo una excelente herramienta didáctica. Sin embargo, los resultados obtenidos en las pruebas de rendimiento (donde se observa cómo el número de operaciones crece exponencialmente conforme el arreglo es más desordenado) confirman que no es una solución viable para el desarrollo de software profesional.

La principal lección que deja este análisis es la importancia de elegir la estructura de datos y el algoritmo adecuado según el contexto. Para arreglos pequeños o con fines educativos, Bubble Sort es suficiente; pero cuando se trata de aplicaciones en producción donde el rendimiento y la escalabilidad son clave, es necesario recurrir a soluciones mucho más optimizadas como Quick Sort o Merge Sort, que garantizan un mejor manejo de la memoria y el tiempo de procesamiento.

Bibliografía

Singh, A. P. (2026, 10 julio). *Bubble sort*. AlgoMaster.io - Master Software Engineering Interviews. https://algomaster.io/learn/dsa/bubble-sort

Bubble Sort — Visualizador de algoritmos. (2026, 21 julio). Algorithm Visualizer. https://alg0.dev/es/bubble-sort/